

Jira Process Automation Plan

ATI Motors — Engineering Analysis & Autonomy Projects *Date: June 2, 2026*

Overview

This document covers three automation initiatives to reduce manual data entry, enforce nomenclature consistency, and surface robot context automatically in Jira bugs.

Part 1: Vehicle Serial Number — Controlled Field Linking (MOM → EA/ANY)

Problem

Support teams manually type Vehicle Serial No in EA/ANY tickets, leading to inconsistent nomenclature (e.g., "10K09" vs "10K 09" vs "10k 09").

Solution Architecture

Step 1 — Create a "Vehicle Serial Number" custom field in EA and ANY projects if the field does not exist

- Field type: **Select List (single choice)** with a managed option set
- Make it **mandatory** on bug/defect issue types
- Maintain a canonical list of vehicle serial numbers (e.g., "10K 09", "PM 11", ...) as the option set

Why not a free-text field? A select list enforces exact nomenclature. The support team picks from a validated list instead of typing.

Step 2 — Keep the options list in sync with MOM

Jira's native select field options are static — they don't auto-sync from another project. Two approaches:

Option A (Recommended): ScriptRunner / Automation for Jira (app)

- Use a scheduled Jira Automation rule (or ScriptRunner Groovy script) that:
 1. JQL-queries all MOM tickets: `project = MOM AND "Vehicle Serial No" is not EMPTY`
 2. Extracts unique values from the Vehicle Serial No field
 3. Calls the Jira REST API to update the allowed values on the EA/ANY custom field
- Run this sync daily or trigger it when a new MOM ticket is created

Option B (Low-tech fallback): Linked issue lookup

- Keep the EA "Vehicle Serial Number" as a free-text field but pair it with a **Jira Automation rule**:
 - Trigger: Issue created or field updated in EA/ANY
 - Action: Validate the entered value against existing MOM "Vehicle Serial No" values via a JQL lookup
 - If not matched → add a comment flagging the mismatch and assign to reporter for correction
- This doesn't prevent wrong input but catches it immediately

Permissions Required

Permission	Where
Jira Admin (or Project Admin + Custom Field Admin)	To create and configure the custom select field
Automation for Jira (project or global)	To run sync and validation rules
ScriptRunner (if used)	Jira admin or ScriptRunner admin role
Browse Projects	EA, ANY, MOM — for cross-project JQL in automations

Part 2: Git Release Tags → Jira Fix Versions & Affects Versions

Problem

- Fix Versions and Affects Versions are manually maintained or missing
- No historical versions exist for issues filed before automation

Solution Architecture

Step 1 — Webhook from GitHub/GitLab on tag push

When a release tag is pushed (e.g., `mule_v2_amz_2026-05-29`):

None

GitHub/GitLab → webhook → Jira Automation (or middleware) → Create Version + Update issues

Option A: Jira Automation (native, no code)

- Trigger: Incoming webhook from GitHub on `create` event with `ref_type: tag`
- Action 1: Call Jira REST API via "Send web request" action to create the version:

None

```
POST /rest/api/3/version
{ "name": "{{tag_name}}", "project": "EA", "released": true, "releaseDate":
  "{{tag_date}}" }
```

- Action 2: JQL-search open issues for the component and set Fix Version

Option B: GitHub Actions workflow (Recommended for reliability)

None

```
on:
  push:
    tags: ['*']

jobs:
  sync-jira:
    steps:
      - name: Create Jira version
        run: |
          curl -X POST https://ati-motors.atlassian.net/rest/api/3/version \
            -H "Authorization: Basic ${ secrets.JIRA_TOKEN }" \
            -d '{"name":"${ github.ref_name }","project":"EA","released":true}'
```

- This is more reliable, auditable, and handles errors better than a Jira automation webhook parse

Step 2 — Set Affects Version on bug-type issues

This works in three stages: bootstrap, ongoing commit linking, and release-time Fix Version population.

2a — Bootstrap: Create the first Affects Version manually

Before any automation exists, create the current validation cycle version by hand in Jira:

- In EA (and ANY) project: **Project Settings** → **Versions** → **Create Version**
- Name it with your validation cycle convention, e.g. `va1-26.6` (year.month or sprint-based)
- Set start date to the beginning of the current validation cycle
- Mark it **unreleased** — it becomes the "currently under test" baseline
- This is a one-time action; all subsequent versions are created automatically by the git tag workflow in Step 1

All bugs filed from this point forward should have `va1-26.6` set as their Affects Version. A Jira Automation rule can default-set Affects Version to the latest unreleased version on issue creation under the same epic so reporters don't have to choose it manually.

2b — Git commit ↔ Jira ID linking (ongoing)

Every valid bug fix must have a traceable git commit. Enforce this at two points:

Enforcement: branch naming or PR title convention

Require Jira issue key in the branch name or PR title:

None

branch: `EA-123/fix-navigation-timeout`

PR title: `EA-123: Fix navigation timeout on pallet mover`

Add a GitHub Actions check that fails the PR if no Jira key is detected:

None

on: `[pull_request]`

jobs:

check-jira-ref:

runs-on: `ubuntu-latest`

steps:

- name: `Verify Jira key in PR title`

```

run: |
  echo "${{ github.event.pull_request.title }}" | grep -E '[A-Z]+-[0-9]+'
\
  || (echo "PR title must contain a Jira issue key (e.g. EA-123)" && exit
1)

```

Auto-linking commits into Jira via Smart Commits

Atlassian's built-in Smart Commits automatically link GitHub commits to Jira issues when the commit message contains the issue key:

```

None
git commit -m "EA-123: Fix navigation timeout on pallet mover"

```

This populates the **Development** panel in the Jira issue with the commit, branch, and PR — no extra tooling needed beyond connecting your GitHub repo to Jira (via Jira's GitHub integration under **Project Settings** → **Toolchain**).

For non-valid bugs (closed as "Won't Fix", "Duplicate", "Not a Bug"), the workflow exit removes the Fix Version requirement — no commit needed. Valid bugs that are resolved without a commit should be blocked by a **Jira workflow validator**: transition to "Done" is gated on the Development panel showing at least one linked commit.

2c — Fix Version population on release tag

When a release tag is pushed (e.g., [GA26.6.1-release](#)), a GitHub Actions workflow:

1. **Creates the Jira version** for the tag (already covered in Step 1)
2. **Queries all resolved bugs under the Epic** via Jira REST:

```

None
JQL: project in (EA, ANY) AND issuetype = Bug AND status = Done
    AND "Epic Link" = <epic-key> AND fixVersion is EMPTY

```

3. **For each bug**, checks whether its linked commit SHA is present in the release tag:

```

Shell
git tag --contains <commit_sha> | grep -q "GA26.6.1-release"

```

4. **If commit is in the release** → sets Fix Version to the release tag via Jira REST:

None

```
PUT /rest/api/3/issue/{issueKey}
{ "update": { "fixVersions": [{ "add": { "name": "GA26.6.1-release" } }] } }
```

5. **If commit is missing from the release** → posts a comment on the Jira issue:

None

⚠ Commit <sha> for this fix was NOT found in val-26.6-release.
Fix Version has not been set. Verify if the fix was cherry-picked or deferred.

This surfaces gaps before the release is marked as done, rather than discovering them in the next validation cycle.

Full GitHub Actions workflow:

None

```
on:
  push:
    tags: ['val-*']

jobs:
  sync-fix-versions:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
        with:
          fetch-depth: 0 # full history needed for git tag --contains

      - name: Create Jira version for tag
        run: |
          curl -X POST https://ati-motors.atlassian.net/rest/api/3/version \
            -H "Authorization: Basic ${ secrets.JIRA_TOKEN }" \
            -H "Content-Type: application/json" \
            -d '{"name": "${ github.ref_name }", "project": "EA", "released": true}'

      - name: Set Fix Versions on resolved bugs
        env:
          JIRA_TOKEN: ${ secrets.JIRA_TOKEN }
          TAG: ${ github.ref_name }
        run: python3 .github/scripts/sync_fix_versions.py
```

The Python script `sync_fix_versions.py` handles the JQL query, commit presence check, and REST updates — ~80 lines, maintainable without Jira admin access after initial setup.

Step 3 — Backfill historical Affects Versions

- **Manual**

Backfill script can be run by a Jira admin with API token — no ScriptRunner needed.

Permissions Required

Permission	Where
Jira API token (admin or project-scoped)	To create versions and update issues via REST
GitHub/GitLab: repo write + CI/CD secrets	To configure GitHub Actions or webhook
"Administer Projects" in EA, ANY	To create Fix/Affects Versions
Automation for Jira	If using native Jira automation for webhook ingestion

Part 3: Mule Bug Auto-Population from `info.txt`

Context

When a Mule bug is filed against a robot, the `info.txt` file on the robot contains key diagnostic context:

```
None
sherpa: pm-v22-02 192.168.7.28
vehicle_type (push time): pallet_mover_amz
vehicle_type (run time): pallet_mover
board_type: advantech
software_tag: mule_v2_amz
installed_date: Fri 29 May 2026 01:51:32 PM IST
commit_date: 2026-05-29 05:39:44 +0530
commit_id: 9e9ec29b745f95dcc51097a40e14a2f06f705d1a
```

Custom Fields to Add on EA/ANY (Mule bug type)

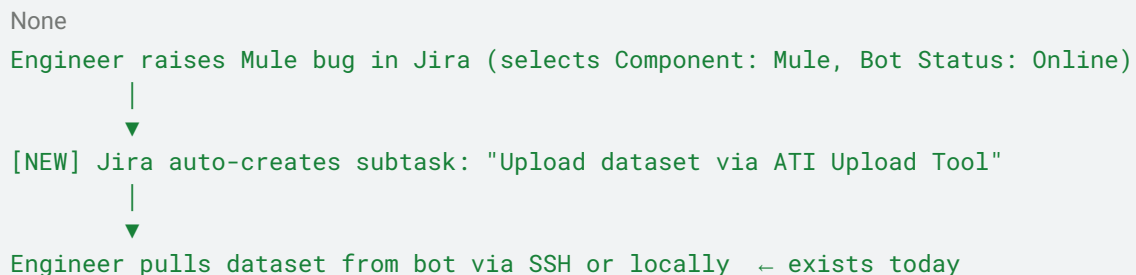
Jira Field Name	Maps to	Type
Sherpa ID	sherpa (e.g., pm-v22-02)	Text
Robot IP	sherpa IP part	Text
Vehicle Type (Push)	vehicle_type (push time)	Select
Vehicle Type (Run)	vehicle_type (run time)	Select
Board Type	board_type	Select
Software Tag	software_tag	Text
Installed Date	installed_date	Date
Commit ID	commit_id	Text
Commit Date	commit_date	DateTime

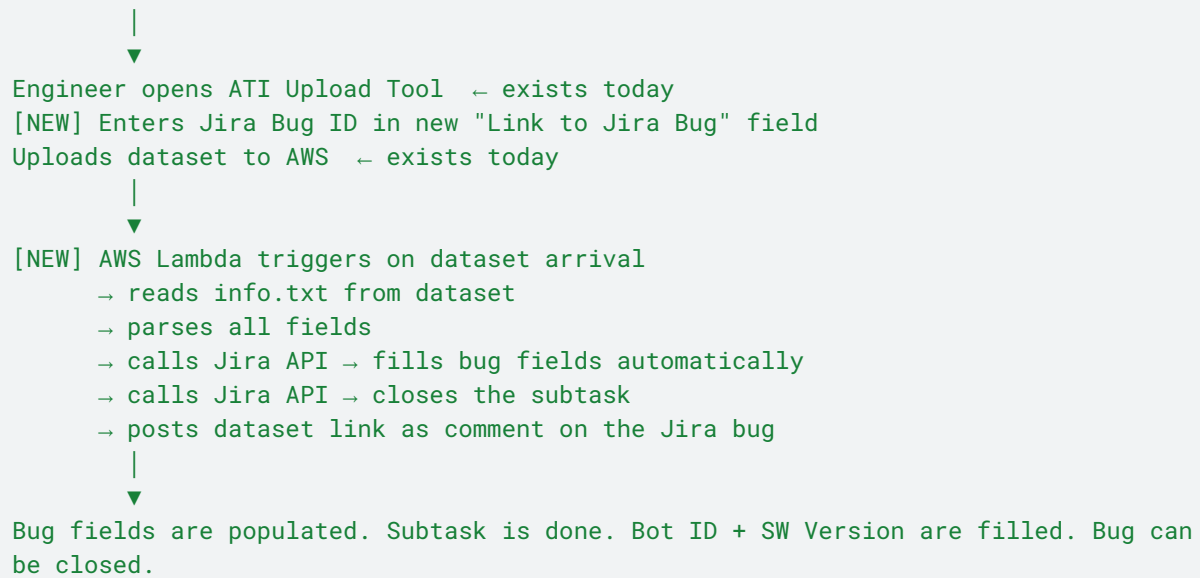
Solution: ATI Upload Tool Integration

We already have an internal tool called **ATI Upload Tool** that service and support engineers use to upload datasets pulled from bots (via SSH at the customer site, or locally). Rather than creating a separate Jira attachment step, we hook into this existing workflow with small additions.

Everything marked **[NEW — to be built]** is net-new automation. Everything else exists today.

How It Works: Bot is Online





The bug is not blocked at any point. Engineers can triage and fix in parallel while the dataset upload happens in the background.

What Needs to Be Built

A. ATI Upload Tool — UI Change [NEW]

Add one new field to the upload form:

- **Label:** "Link to Jira Bug (optional)"
- **Type:** Text input, accepts Jira issue key format (e.g., **EA-456**, **ANY-789**)
- **Behaviour:** If filled, the Jira Bug ID is stored as metadata on the S3 object. If left blank, engineer can paste the dataset link into Jira manually instead (see Two-Way Linkage below).

No other changes to the upload flow.

B. ATI Upload Tool — Backend Change [NEW]

After a successful upload, if a Jira Bug ID was provided, the tool backend makes one Jira API call to post a comment on that issue confirming the upload:

None

Comment posted automatically on Jira bug:

"Dataset uploaded via ATI Upload Tool.

Dataset ID: ds-20260529-pm-v22-02

Bot info will be auto-populated shortly once the server processes the upload."

This gives the reporter immediate confirmation, before the Lambda finishes parsing.

Requires: ATI Upload Tool backend holds a **Jira API token** (stored as an environment secret, not hardcoded).

C. AWS Server — New Lambda Function [NEW]

This is the core new automation. A small AWS Lambda function is triggered automatically when a new dataset lands in S3 (S3 ObjectCreated event). It does four things in sequence:

1. Read the dataset and extract info.txt

Python

```
def handler(event, context):
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = event['Records'][0]['s3']['object']['key']

    obj = s3.get_object(Bucket=bucket, Key=key)
    jira_bug_id = obj['Metadata'].get('jira-bug-id') # stored by ATI Upload Tool

    # Dataset is a .tar.gz archive - extract info.txt
    import tarfile, io
    with tarfile.open(fileobj=io.BytesIO(obj['Body'].read())) as tar:
        info_raw = tar.extractfile('data/current/info.txt').read().decode()
```

2. Parse info.txt fields

Python

```
fields = {}
for line in info_raw.splitlines():
    if ':' in line:
        k, _, v = line.partition(':')
        fields[k.strip()] = v.strip()

parsed = {
    'Sherpa ID': fields.get('sherpa', '').split()[0],
    'Robot IP': fields.get('sherpa', '').split()[-1],
```

```

'Vehicle Type (Push)': fields.get('vehicle_type (push time)'),
'Vehicle Type (Run)':  fields.get('vehicle_type (run time)'),
'Board Type':          fields.get('board_type'),
'Software Tag':         fields.get('software_tag'),
'Installed Date':       fields.get('installed_date'),
'Commit ID':            fields.get('commit_id'),
'Commit Date':          fields.get('commit_date'),
}

```

3. Update the Jira bug with parsed fields

Python

```

if not jira_bug_id:
    return # no bug linked, skip Jira update

# Jira custom field IDs are set once by Jira admin, stored in Lambda env vars
jira_payload = {
    "fields": {
        FIELD_SHERPA_ID:    parsed['Sherpa ID'],
        FIELD_ROBOT_IP:     parsed['Robot IP'],
        FIELD_VTYPE_PUSH:   {"value": parsed['Vehicle Type (Push)']},
        FIELD_VTYPE_RUN:    {"value": parsed['Vehicle Type (Run)']},
        FIELD_BOARD_TYPE:   {"value": parsed['Board Type']},
        FIELD_SW_TAG:        parsed['Software Tag'],
        FIELD_INSTALLED:     parsed['Installed Date'],
        FIELD_COMMIT_ID:     parsed['Commit ID'],
        FIELD_COMMIT_DATE:   parsed['Commit Date'],
    }
}
requests.put(
    f"https://ati-motors.atlassian.net/rest/api/3/issue/{jira_bug_id}",
    json=jira_payload,
    headers={"Authorization": f"Basic {JIRA_TOKEN}"})

```

4. Close the subtask and post dataset link as comment

Python

```

# Find the upload subtask under this bug and transition it to Done
issue = requests.get(
    f"https://ati-motors.atlassian.net/rest/api/3/issue/{jira_bug_id}",
    headers={"Authorization": f"Basic {JIRA_TOKEN}"})
issue.json()

```

```

for subtask in issue['fields']['subtasks']:
    if 'Upload dataset' in subtask['fields']['summary']:
        requests.post(

f"https://ati-motors.atlassian.net/rest/api/3/issue/{subtask['id']}/transitions",
    json={"transition": {"id": DONE_TRANSITION_ID}},
    headers={"Authorization": f"Basic {JIRA_TOKEN}"})

# Post dataset S3 link as comment on the bug
dataset_url = f"https://s3.ati-internal/datasets/{key}"
requests.post(

f"https://ati-motors.atlassian.net/rest/api/3/issue/{jira_bug_id}/comment",
    json={"body": {"type": "doc", "version": 1, "content": [{"type":
"paragraph",
    "content": [{"type": "text",
        "text": f"✅ Bot info auto-populated from dataset.\nDataset:
{dataset_url}"}]}]}},
    headers={"Authorization": f"Basic {JIRA_TOKEN}"})

```

Lambda requires: S3 read access, Jira API token (stored in AWS Secrets Manager), Jira custom field IDs (stored in Lambda environment variables — filled in once by a Jira admin after the custom fields are created).

D. Jira — Two Automation Rules [NEW]

Rule 1: Auto-create subtask on Mule bug creation

Setting	Value
Trigger	Issue Created in EA or ANY
Condition	Component = Mule AND "Bot Status" field = Online
Action	Create sub-task: summary = "Upload dataset via ATI Upload Tool", assignee = reporter

Rule 2: Require Bot ID and SW Version before bug closure

Setting	Value
Trigger	Transition to "Done" attempted on parent bug
Condition	"Bot ID" field is empty OR "SW Version" field is empty
Action	Block transition, show: <i>"Please fill in Bot ID and SW Version before closing this bug."</i>

This is the only gate for closure. The upload subtask status does **not** block closing — it is informational only. If the ATI Upload Lambda runs successfully, it fills Bot ID (Sherpa ID) and SW Version (Software Tag) automatically, so the bug closes without any manual action. If the upload didn't happen or failed, the engineer simply types the two fields manually and closes normally.

Both rules are configured under **Project Settings** → **Automation** in EA and ANY. They require project-admin access to set up.

Also needed: a **"Bot Status" field** (Radio button: Online / Offline) added to the Mule bug creation screen — this drives Rule 1 (subtask creation) and the offline flow below.

Two-Way Linkage

Both directions work independently, covering different field scenarios:

Scenario	How the link is made
Bug ID known at upload time	Engineer types Jira Bug ID into ATI Upload Tool → Lambda auto-fills Jira and closes subtask
Upload done before bug is created	Engineer copies the dataset ID/S3 link from ATI Upload Tool → pastes into the Jira bug's "Dataset Link" field manually
Bug created first, upload follows later	The auto-created subtask reminds the engineer; they refer to the bug ID when opening the Upload Tool

The "Dataset Link" field in Jira is a plain one-line text field added to the Mule bug screen — for cases where the engineer needs to paste the upload reference manually rather than going through the automated path.

Fallback: Bot is Offline

When the bot is switched off or unreachable and no dataset can be pulled:

- Engineer selects **Bot Status: Offline** when creating the bug
- **No subtask is created** — the upload workflow is skipped entirely
- Engineer fills in **Bot ID** and **SW Version** manually at creation time or any point before closing (from the physical robot label, MOM ticket, or Sanjaya deployment record)
- Once those two fields are filled, the bug can be closed normally — the same Rule 2 validator applies

No separate offline process. Same two fields, same closure rule as the online path.

Summary: Required Permissions

Role / Access	Needed For
Jira Administrator	Create custom fields (Vehicle Serial No, Bot Status, Bot ID, SW Version, Dataset Link, all Mule info fields), configure field screens
Project Administrator (EA, ANY, MOM)	Add fields to project screens, create automation rules (subtask creation, Done validator), manage Fix/Affects Versions
Automation for Jira (global)	Cross-project automations — MOM → EA serial sync, subtask rules, workflow validators
Jira API Token (admin or project-scoped)	Used by AWS Lambda to update bug fields, close subtasks, post comments. Also used by GitHub Actions for version sync and ATI Upload Tool backend for upload confirmation comments

Role / Access	Needed For
AWS IAM — Lambda execution role	S3 read access on the datasets bucket; permission to read object metadata; access to AWS Secrets Manager (for Jira token)
AWS Secrets Manager	Stores Jira API token securely for use by the Lambda function
ATI Upload Tool — backend deploy access	To add the Jira Bug ID field to the upload form and the post-upload Jira comment call
ScriptRunner license (optional)	Dynamic field option sync from MOM Vehicle Serial No values (Part 1, Option A)